Prise de notes Cours Unix pour révision : Environnement de travail

Derrez Lorenzo

October 14, 2026

Ordinateur

- Machine en couche $\Longrightarrow$ couche matérielle, couche d'application couche système d'exploitation
- Consituée de :

  processeurs : sert à executer les instructions du programme

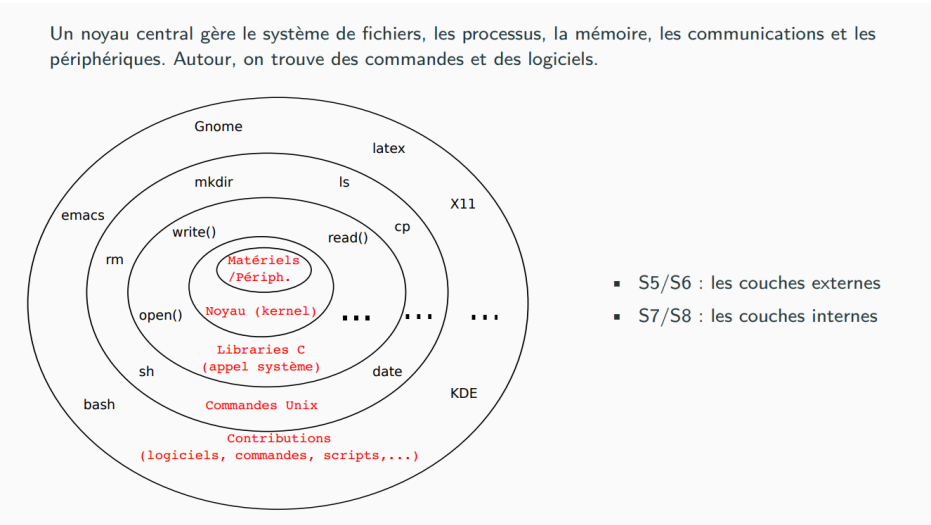
  mémoire centrale : stock le programme en exécution

  périphériques : ce qui permet d'échanger avec l'extérieur.

Système d'exploitation

- logiciel qui fait le lien entre le matériel, les applications et les utilisateurs.
- **SE mono-processus, mono-utilisateur** : exécute un programme à la fois avec un seul utilisateur.
- **SE multi-processus, mono-utilisateur** : plusieurs tâches simultanées correspondant à des programmes différents. Répartition des ressources à tour de rôle entre les applications.
- **SE multi-processus, multi-utilisateurs** : Plusieurs utilisateurs peuvent lancer en même temps leurs programmes sur la même machine.

Organisation générale



Compte utilisateur & protection

- Système multi-processus $\Longrightarrow$ création d'un compte
- **Remarque :** Être responsable sur la gestion de ses mots de passe.

A propos du Shell

- **Shell** : intepréteur de commande pour interagir avec le noyau.
- **interpréteur de commande** : fournit une interface texte entre l'utilisateur et le noyau.
- **bash** : programme shell par défaut dans la plupart des environnements.

Commandes Unix

REMARQUE : cette liste sera ajoutée au fur et à mesures des révisions :

(a) Obtenir de l'aide pour une commande :
    `man` nom_de_la_commande

(b) Enter dans le sous-dossier *divers* du sous-dossier *autres* :
    `cd` autres/divers

(c) Savoir dans quel dossier on se trouve :
    `pwd`

(d) Lister le contenu du dossier courant :
    `ls`
    ls -la permet d'afficher plus de détails sur le contenu

(e) Créer un répertoire :
    `mkdir`

(f) Créer un fichier :
    `touch`

(g) Copier (fichier ou dossier) :
    `cp, cp-r`

(h) Déplacer/Renommer :
    `mv`

(i) Déterminer le type d'un contenu :
    `file`

(j) Afficher un fichier :
    `cat`

(k) Parcourir un fichier :
    `less`

(l) Compter (ligne, lettre, mot, etc...) :
    `wc, wc -l, wc -w`

(m) Supprimer un contenu :
    `rm`

(n) Modifier les droits :
    `chmod`

A propos des droits de fichiers :

r : droit de lecture

- Si répertoire, consultation de ses entrées (c.-à-d. `ls` autorisé)
- Sinon, consultation du contenu du fichier

w : droit d'écriture

- Si répertoire, droit de création, de renommage et de suppression d'une entrée dans le répertoire
- Sinon, droit de modification du contenu du fichier

x : droit d'exécution ou de traversée

- si répertoire, droit de traverser (c.-à-d. `cd` autorisé)
- sinon, droit d'exécution

3 catégories d'utilisateurs :

- Propriétaires (u : user)
- Groupe propriétaire (g : group)
- Tous les autres (o : other)

Pour l'affichage, on a à la suite :

- \- $\Longrightarrow$ Type de fichier
- \- - - $\Longrightarrow$ Droits du propriétaire (rwx)
- \- - - $\Longrightarrow$ Droits du groupe (rwx)
- \- - - $\Longrightarrow$ Droits des autres (rwx)

- Modification des droits sur un fichier/répertoire existant
  - `chmod droit fichier`
- Droits à appliquer :
  - Catégorie : u (=user), g (=group), o(=other), ou a (=all)
  - Opérations : + (ajout), - (retrait), = (affectation)
  - Exemple : u+rx,o=r
- Alternativement, on peut aussi les donner en octal : 4 (lecture), 2 (écriture) et 1 (exécution)
  Exemple : 641 pour rw-r—-x, 755 pour rwxr-xr-x

Raccourcis Shell

(a) Recherche avec motif : C-r

(b) Commande précédente : C-p

(c) Commande suivante : C-n

(d) Historique des commandes : history

(e) Couper/mémoriser le texte depuis la position du curseur jusqu'à la fin de la ligne : C-k

(f) Insérer le texte précédent supprimé : C-y

(g) Début de ligne : C-a

(h) Fin de ligne : C-e

(i) Activer la complétion : tab

(j) Pour contrôler le processus courant :

Lancer en tâche de fond : cmd &

Endormir le processus en cours : C-z

terminer le processus en cours : C-c

Quitter le shell en cours : C-d

(k) Pour manipuler les jobs du shell :

Lister les jobs connus du shell : jobs

Réveiller un job en tâche de fond : bg n

Remettre un job au premier plan : fg n

(l) Obtenir la liste des processus : ps

(m) Obtenit le PID de la dernière commande lancée en tâche de fond : echo &

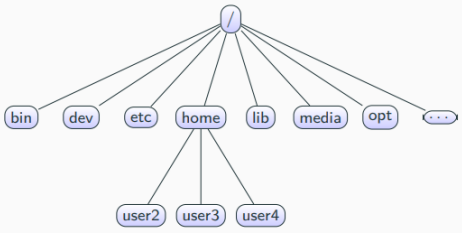
(n) Demander l'arrêt d'un processus spécifique : `kill pid`

(o) Forcer l'arrêt d'un processus spécifique : `kill -KILL pid`

Arborescence et système de fichiers

- Les fichiers sont structurés sous la forme d'un arbre.
- Noeuds $\Longrightarrow$ répertoire.
- Feuilles $\Longrightarrow$ fichiers réguliers.

Arborescence standard :

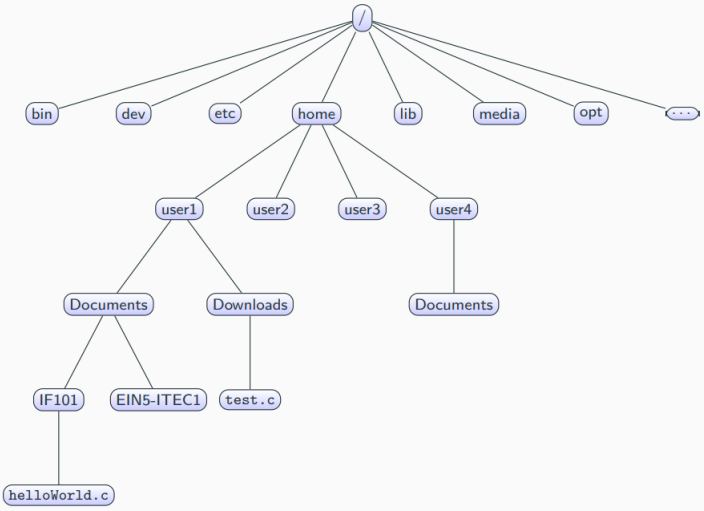


ATTENTION: ici, les utilisateurs sont dans un répertoire `/net/i/` (au lieu de `/home`).

Chemins absolus :

Le point de départ **ne dépend pas de la position courante** (working directory) dans le système de fichier. Il est défini quel que soit l'endroit où on se trouve dans l'arborescence. Il commence par :

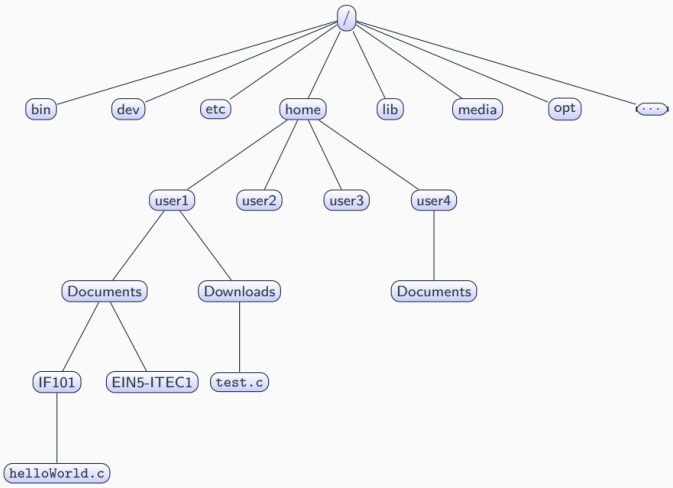
- `/` : La racine du système de fichier
- `~` : Alias/Raccourci vers le répertoire home de l'utilisateur courant
- `~toto` : Alias/Raccourci vers le répertoire home de l'utilisateur `toto`



Chemins relatifs :

Le point de départ **dépend de la position courante** (working directory) dans le système de fichier. Il n'est défini et valide que pour la position courante dans l'arborescence. Il commence par :

- `./` : Le répertoire courant
- `../` : Le répertoire parent
- `subdir1` : un fichier contenu dans le répertoire courant (équivalent à `./subdir`)



Substitutions et variables

Le shell analyse la ligne de commande avant de l'exécuter pour faire des substitutions et des expansions.

Expansion des accolades

`a{b,c,d}e` devient `abe ace ade`

Expansion des tildes

`~` → chemin vers votre répertoire personnel
`~toto` → chemin vers le répertoire personnel de toto

Expansion des variables

- syntaxe d'une variable : `$VAR`, `${VAR}`, `${VAR:-defaut}`
- affectation d'une variable :
  `export VAR=value` → variable disponible dans l'environnement des processus enfants
  `VAR=value` → variable disponible **uniquement** dans l'environnement du processus courant
  Attention PAS d'espace entre le nom de variable et le =

Substitution de commandes

`$( cmd )` exécute `cmd` dans un sous-shell et remplace `$( cmd )` par le résultat de l'exécution de la commande.

Coupure des mots

Chaque séquence de blancs (espaces, tabulations, retours de ligne) délimite les mots-clés de la ligne de commande : commande, paramètres, options, variables, ...

Point important

- `'une chaîne de caractères'` : le contenu n'est pas interprété
- `"une chaîne de caractères"` : le contenu est interprété

Expansion des noms de fichiers

Le shell recherche les caractères spéciaux suivants avant d'exécuter la commande : `*`, `?`, `[...]`.

- `*` : n'importe quelle chaîne de caractères, même vide ;
- `?` : n'importe quel caractère ;
- `[...]` : n'importe quel caractère spécifié entre crochets.

Dans les crochets, on peut indiquer :

- des caractères seuls ;
- des ensembles de caractères séparés par un - (si - est le premier caractère, il désigne le caractère - lui-même).

Exemple : `[04a-dG-L6]` représente l'ensemble des caractères composé de : $0, 4, a, b, c, d, G, H, I, J, K, L, 6$

Protection

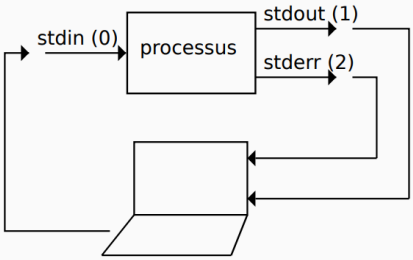
Le caractère d'échappement \ protège un caractère spécial lorsqu'il ne doit pas être évalué.
Par exemple : `mkdir my\ dir`

Mécanismes d'entrées/sorties

Généralement une opération d'E/S :

- transfert de données entre une zone mémoire et un fichier ou un périphérique
- Ex. un processus effectue des E/S : depuis le clavier vers l'écran

À sa création, chaque processus possède 3 flots E/S standards.



- `stdin` (n^0 0) : entrée standard
  opérations lecture, par défaut le clavier
- `stdout` (n^0 1) : sortie standard
  opérations d'écriture, par défaut l'écran
- `stderr` (n^0 2) sortie d'erreur
  opérations d'affichage des messages d'erreurs, par défaut l'écran

Unix permet à l'utilisateur de modifier et de reconnecter les flots d'E/S
c'est le mécanisme de redirection

Redirections courantes

- `< file` : redirection de l'entrée standard depuis un fichier
- `> file` : redirection de la sortie standard vers un fichier
- `>> file` : redirection en ajoutant à la fin du fichier
- `cmd | cmd` : pipe entre deux commandes

Sortie d'erreur

- `2> file` : redirige la sortie d'erreur vers `file`
- `2>&1` : fusionne la sortie d'erreur avec la sortie standard